```
# === File: _start_api.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
import os, subprocess, sys
from pathlib import Path
env = os.environ.copy()
p = Path("backend/.env.docker") if Path("backend/.env.docker").exists() else Path(".env.docker")
for line in p.read_text(encoding="utf-8").splitlines():
line=line.strip()
if not line or line.startswith("#") or "=" not in line: continue
k,v=line.split("=",1)
env[k.strip()]=v.strip()
pu=env.get("POSTGRES_USER","agentflow")
pp=env.get("POSTGRES_PASSWORD","")
pd=env.get("POSTGRES_DB","agentflow_eval")
env["DATABASE_URL"]=f"postgresql+asyncpg://{pu}:{pp}@127.0.0.1:5432/{pd}"
env["REDIS_URL"]="redis://127.0.0.1:6379/0"
env["CELERY_BROKER_URL"]=env["REDIS_URL"]
env["CELERY_RESULT_BACKEND"]=env["REDIS_URL"]
env["CELERY_TASK_ALWAYS_EAGER"]="true"
env["TASK_QUEUE_BACKEND"]="eager"
env["DEPLOY_PROFILE"]="lite"
env["ENV"]="dev"
env["DEBUG"]="true"
env["AUTH_ENABLED"]="false"
env["BILLING_ENABLED"]="false"
env["LOG_DB_SINK"]="true"
env["PYTHONPATH"]=str(Path(".").resolve())
os.chdir(str(Path(".").resolve()))
log=open("uvicorn_docker_db.log","w",encoding="utf-8")
proc=subprocess.Popen([sys.executable,"-m","uvicorn","app.main:app","--host","127.0.0.1","--port","8000"], e..
print(proc.pid)
# === File: app/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""AgentFlow-Eval 后端应用包。"""
# === File: app/api/__init__.py ===
# AgentFlow-Eval V1.0.0
"""Application package."""
# === File: app/api/v1/__init__.py ===
# AgentFlow-Eval V1.0.0
"""Application package."""
# === File: app/api/v1/endpoints/__init__.py ===
# AgentFlow-Eval V1.0.0
"""Application package."""
# === File: app/api/v1/endpoints/ab.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Online A/B testing REST API."""
from __future__ import annotations
from typing import Any
```

```
from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload
from app.core.ab import service as ab_service
from app.core.ab.service import utcnow
from app.core.audit import write_audit
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.models.ab_test import ABExperiment, ABStatus, ABVariant
from app.schemas.ab_test import (
ABAssignRequest,
ABAssignResponse,
ABExperimentCreate,
ABExperimentListResponse,
ABExperimentResponse,
ABSampleSizeRequest,
ABStatusUpdate,
ABTrackRequest,
ABVariantCreate,
ABVariantResponse,
)
from app.utils.exceptions import BusinessError, NotFoundError
router = APIRouter()
def _actor(request: Request) -> str:
return getattr(request.state, "actor", None) or "anonymous"
def _to_response(exp: ABExperiment) -> ABExperimentResponse:
return ABExperimentResponse(
id=exp.id,
key=exp.key,
name=exp.name,
description=exp.description or "",
status=exp.status,
alpha=float(exp.alpha or 0.05),
min_sample_size=int(exp.min_sample_size or 100),
primary_metric=exp.primary_metric or "conversion",
control_variant_key=exp.control_variant_key,
source_experiment_id=exp.source_experiment_id,
config=exp.config or {},
created_by=exp.created_by or "anonymous",
started_at=exp.started_at,
ended_at=exp.ended_at,
winner_variant_key=exp.winner_variant_key,
created_at=exp.created_at,
variants=[
ABVariantResponse(
id=v.id,
key=v.key,
name=v.name or v.key,
weight=float(v.weight or 1),
```

```
is_control=bool(v.is_control),
payload=v.payload or {},
description=v.description or "",
)
for v in (exp.variants or [])
],
)
@router.post("", response_model=ABExperimentResponse, status_code=201)
@require_permission(Permission.TASK_CREATE)
async def create_ab_experiment(
body: ABExperimentCreate,
request: Request,
session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
"""Create an online A/B experiment with ≥2 variants."""
actor = _actor(request)
existing = (
await session.execute(select(ABExperiment).where(ABExperiment.key == body.key))
).scalar_one_or_none()
if existing:
raise BusinessError(f"Experiment key already exists: {body.key}")
controls = [v for v in body.variants if v.is_control]
control_key = controls[0].key if controls else body.variants[0].key
exp = ABExperiment(
key=body.key,
name=body.name,
description=body.description or "",
status=ABStatus.RUNNING.value
if body.start_immediately
else ABStatus.DRAFT.value,
alpha=body.alpha,
min_sample_size=body.min_sample_size,
primary_metric=body.primary_metric,
control_variant_key=control_key,
source_experiment_id=body.source_experiment_id,
config=body.config or {},
created_by=actor,
started_at=utcnow() if body.start_immediately else None,
)
session.add(exp)
await session.flush()
for i, v in enumerate(body.variants):
is_ctrl = v.is_control or (not controls and i == 0)
session.add(
ABVariant(
experiment_id=exp.id,
key=v.key,
name=v.name or v.key,
weight=v.weight,
is_control=is_ctrl,
```

```
payload=v.payload or {},
description=v.description or "",
)
)
await write_audit(
session,
action="ab.create",
resource_type="ab_experiment",
resource_id=exp.id,
actor=actor,
detail={"key": body.key, "variants": [v.key for v in body.variants]},
)
await session.commit()
exp = await ab_service.get_experiment_by_id(session, exp.id)
return _to_response(exp)
@router.get("", response_model=ABExperimentListResponse)
@require_permission(Permission.TASK_READ)
async def list_ab_experiments(
request: Request,
page: int = Query(1, ge=1),
page_size: int = Query(20, ge=1, le=100),
status: str | None = Query(None),
session: AsyncSession = Depends(get_db),
) -> ABExperimentListResponse:
actor = _actor(request)
q = select(ABExperiment).options(selectinload(ABExperiment.variants))
cq = select(func.count(ABExperiment.id))
from app.core.tenancy import is_admin, tenancy_enforced
if tenancy_enforced() and not is_admin(actor):
q = q.where(ABExperiment.created_by == actor)
cq = cq.where(ABExperiment.created_by == actor)
if status:
q = q.where(ABExperiment.status == status)
cq = cq.where(ABExperiment.status == status)
total = int((await session.execute(cq)).scalar() or 0)
rows = (
(
await session.execute(
q.order_by(ABExperiment.created_at.desc())
.offset((page - 1) * page_size)
.limit(page_size)
)
)
.scalars()
.all()
)
return ABExperimentListResponse(
items=[_to_response(r) for r in rows],
total=total,
page=page,
```

```
page_size=page_size,
)
@router.post("/sample-size")
@require_permission(Permission.TASK_READ)
async def sample_size(
body: ABSampleSizeRequest,
request: Request,
) -> dict[str, Any]:
"""Recommend per-variant sample size for conversion-rate A/B tests."""
return await ab_service.recommend_sample_size(
baseline_rate=body.baseline_rate,
mde=body.mde,
alpha=body.alpha,
power=body.power,
)
@router.post(
"/from-offline/{experiment_id}",
response_model=ABExperimentResponse,
status_code=201,
)
@require_permission(Permission.TASK_CREATE)
async def create_ab_from_offline_experiment(
experiment_id: str,
request: Request,
key: str = Query(..., min_length=1, max_length=100),
session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
"""Promote an offline Experiment's runs into an online A/B draft."""
from app.models.experiment import Experiment
actor = _actor(request)
offline = (
await session.execute(
select(Experiment)
.options(selectinload(Experiment.runs))
.where(Experiment.id == experiment_id)
)
).scalar_one_or_none()
if offline is None:
raise NotFoundError("Experiment", experiment_id)
if not offline.runs or len(offline.runs) < 2:
raise BusinessError("Offline experiment needs at least 2 runs to create A/B")
existing = (
await session.execute(select(ABExperiment).where(ABExperiment.key == key))
).scalar_one_or_none()
if existing:
raise BusinessError(f"Experiment key already exists: {key}")
variants = [
ABVariantCreate(
key=r.label,
name=r.label,
```

```
weight=1.0,
is_control=(i == 0),
payload={"agent_config": r.agent_config or {}},
)
for i, r in enumerate(offline.runs)
]
exp = ABExperiment(
key=key,
name=f"AB from {offline.name}",
description=f"Promoted from offline experiment {offline.id}",
status=ABStatus.DRAFT.value,
control_variant_key=variants[0].key,
source_experiment_id=offline.id,
created_by=actor,
config={},
)
session.add(exp)
await session.flush()
for i, v in enumerate(variants):
session.add(
ABVariant(
experiment_id=exp.id,
key=v.key,
name=v.name,
weight=v.weight,
is_control=v.is_control or i == 0,
payload=v.payload,
)
)
await session.commit()
exp = await ab_service.get_experiment_by_id(session, exp.id)
return _to_response(exp)
@router.get("/{experiment_id}", response_model=ABExperimentResponse)
@require_permission(Permission.TASK_READ)
async def get_ab_experiment(
experiment_id: str,
request: Request,
session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
exp = await ab_service.get_experiment_by_id(session, experiment_id)
return _to_response(exp)
@router.patch("/{experiment_id}/status", response_model=ABExperimentResponse)
@require_permission(Permission.TASK_UPDATE)
async def update_ab_status(
experiment_id: str,
body: ABStatusUpdate,
request: Request,
session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
exp = await ab_service.get_experiment_by_id(session, experiment_id)
```

```
new_status = body.status
if new_status == ABStatus.RUNNING.value and exp.status != ABStatus.RUNNING.value:
exp.started_at = exp.started_at or utcnow()
exp.ended_at = None
if new_status in {ABStatus.COMPLETED.value, ABStatus.ARCHIVED.value}:
exp.ended_at = utcnow()
exp.status = new_status
await session.commit()
exp = await ab_service.get_experiment_by_id(session, experiment_id)
return _to_response(exp)
@router.post("/{experiment_key}/assign", response_model=ABAssignResponse)
@require_permission(Permission.TASK_READ)
async def assign_unit(
experiment_key: str,
body: ABAssignRequest,
request: Request,
session: AsyncSession = Depends(get_db),
) -> ABAssignResponse:
"""Sticky assign a unit to a variant (and optionally log exposure)."""
payload = await ab_service.assign(
session,
experiment_key=experiment_key,
unit_id=body.unit_id,
context=body.context,
record_exposure=body.record_exposure,
)
return ABAssignResponse(**payload)
@router.post("/{experiment_key}/track")
@require_permission(Permission.TASK_UPDATE)
async def track_event(
experiment_key: str,
body: ABTrackRequest,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Track exposure / conversion / metric for a unit."""
return await ab_service.track_event(
session,
experiment_key=experiment_key,
unit_id=body.unit_id,
event_type=body.event_type,
metric_name=body.metric_name,
metric_value=body.metric_value,
properties=body.properties,
auto_assign=body.auto_assign,
)
@router.get("/{experiment_id}/results")
@require_permission(Permission.EVALUATION_READ)
async def ab_results(
experiment_id: str,
```

```
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Statistical analysis: conversion rates, lift, p-values, optional winner."""
return await ab_service.analyze(session, experiment_id)
# === File: app/api/v1/endpoints/agents_http.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""HTTP Agent product APIs — contract docs + probe (Phase 1)."""
from __future__ import annotations
import time
from typing import Any
import httpx
from fastapi import APIRouter, Request
from pydantic import BaseModel, Field
from app.config import settings
from app.core.agent_runner.protocol import (
PROTOCOL_VERSION,
build_http_request_payload,
coerce_http_response,
failed_http_result,
)
from app.core.agent_runner.ssrf import SsrfBlockedError, validate_http_agent_url
from app.core.rbac import Permission, require_permission
router = APIRouter()
class HttpAgentProbeRequest(BaseModel):
"""Probe an external HTTP agent endpoint."""
endpoint_url: str = Field(..., min_length=1, description="Absolute http(s) URL")
timeout_sec: float = Field(default=10.0, ge=1.0, le=120.0)
headers: dict[str, str] = Field(default_factory=dict)
method: str = Field(default="POST")
query: str = Field(default="ping", description="Probe query string")
context: dict[str, Any] = Field(default_factory=dict)
verify_ssl: bool = True
class HttpAgentProbeResponse(BaseModel):
"""Probe result — always JSON (use ok/reachable for UX, not only HTTP status)."""
ok: bool = False
reachable: bool = False
protocol_compatible: bool = False
ssrf_blocked: bool = False
latency_ms: int | None = None
http_status: int | None = None
endpoint: str = ""
protocol_version: str = PROTOCOL_VERSION
final_answer_preview: str = ""
steps_count: int = 0
normalized_status: str = ""
error: str | None = None
detail: dict[str, Any] = Field(default_factory=dict)
@router.get("/contract")
```

```
@require_permission(Permission.TASK_READ)
async def http_agent_contract(request: Request) -> dict[str, Any]:
"""Return agentflow.http.v1 contract summary for UI / integrators."""
return {
"protocol_version": PROTOCOL_VERSION,
"request": {
"method": "POST",
"content_type": "application/json",
"body": {
"protocol_version": PROTOCOL_VERSION,
"query": "string",
"tools": ["string"],
"context": {},
"meta": {},
},
},
"response_accepted": [
{
"name": "full",
"example": {
"status": "success",
"final_answer": "...",
"steps": [],
"total_tokens": 0,
"response_time_ms": 0,
},
},
{"name": "short", "example": {"answer": "..."}},
{"name": "plain_text", "example": "raw answer body"},
],
"agent_config_fields": {
"runner": "http | http_agent | remote | webhook",
"endpoint_url": "https://agent.example.com/v1/invoke",
"timeout_sec": 60,
"headers": {"Authorization": "Bearer ..."},
"method": "POST",
"context": {},
"verify_ssl": True,
},
"docs": "docs/http-agent-protocol.md",
}
@router.post("/probe", response_model=HttpAgentProbeResponse)
@require_permission(Permission.TASK_CREATE, Permission.TASK_EXECUTE)
async def probe_http_agent(
body: HttpAgentProbeRequest,
request: Request,
) -> HttpAgentProbeResponse:
"""Probe an external HTTP agent: SSRF check, reachability, protocol normalize."""
allow_private = bool(getattr(settings, "HTTP_AGENT_ALLOW_PRIVATE_IP", False))
endpoint = (body.endpoint_url or "").strip()
```

```
try:
endpoint = validate_http_agent_url(endpoint, allow_private=allow_private)
except SsrfBlockedError as exc:
return HttpAgentProbeResponse(
ok=False,
reachable=False,
protocol_compatible=False,
ssrf_blocked=True,
endpoint=endpoint or body.endpoint_url,
error=str(exc),
detail={"code": "ssrf_blocked"},
)
method = (body.method or "POST").upper()
payload = build_http_request_payload(
body.query or "ping",
tools=[],
context=body.context or {},
meta={"probe": True},
)
headers = {"Content-Type": "application/json", **(body.headers or {})}
t0 = time.monotonic()
try:
async with httpx.AsyncClient(
timeout=float(body.timeout_sec),
verify=bool(body.verify_ssl),
) as client:
resp = await client.request(
method,
endpoint,
json=payload if method in {"POST", "PUT", "PATCH"} else None,
params=payload if method == "GET" else None,
headers=headers,
)
latency = int((time.monotonic() - t0) * 1000)
except httpx.TimeoutException as exc:
return HttpAgentProbeResponse(
ok=False,
reachable=False,
protocol_compatible=False,
latency_ms=int((time.monotonic() - t0) * 1000),
endpoint=endpoint,
error=f"Timeout after {body.timeout_sec}s: {exc}",
detail={"code": "timeout"},
)
except httpx.HTTPError as exc:
return HttpAgentProbeResponse(
ok=False,
reachable=False,
protocol_compatible=False,
latency_ms=int((time.monotonic() - t0) * 1000),
```

```
endpoint=endpoint,
error=f"HTTP request failed: {exc}",
detail={"code": "http_error"},
)
if resp.status_code >= 400:
normalized = failed_http_result(
query=body.query or "ping",
error=f"HTTP {resp.status_code}: {(resp.text or '')[:300]}",
elapsed_ms=latency,
endpoint=endpoint,
)
return HttpAgentProbeResponse(
ok=False,
reachable=True,
protocol_compatible=False,
latency_ms=latency,
http_status=resp.status_code,
endpoint=endpoint,
error=normalized.get("error_message") or f"HTTP {resp.status_code}",
normalized_status="failed",
detail={"code": "http_status", "body_preview": (resp.text or "")[:200]},
)
ct = (resp.headers.get("content-type") or "").lower()
data: Any
if "application/json" in ct or (resp.text or "").lstrip().startswith(("{", "[")):
try:
data = resp.json()
except ValueError:
data = {"answer": resp.text}
else:
data = {"answer": resp.text or ""}
normalized = coerce_http_response(
data, query=body.query or "ping", elapsed_ms=latency, endpoint=endpoint
)
status = str(normalized.get("status") or "")
final = str(normalized.get("final_answer") or "")
steps = normalized.get("steps") or []
compatible = status in {"success", "max_iterations_reached"} and (
bool(final) or bool(steps)
)
parseable = bool(normalized.get("runner") == "http")
return HttpAgentProbeResponse(
ok=compatible,
reachable=True,
protocol_compatible=compatible or parseable,
latency_ms=latency,
http_status=resp.status_code,
endpoint=endpoint,
final_answer_preview=final[:200],
steps_count=len(steps) if isinstance(steps, list) else 0,
```

```
normalized_status=status,
error=None if compatible else (normalized.get("error_message") or "No usable answer"),
detail={
"code": "ok" if compatible else "protocol_partial",
"total_tokens": normalized.get("total_tokens"),
},
)
# === File: app/api/v1/endpoints/audit.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Audit log query API."""
from __future__ import annotations
from typing import Any
from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.models.audit_log import AuditLog
router = APIRouter()
@router.get("")
@require_permission(Permission.AUDIT_READ)
async def list_audit_logs(
request: Request,
page: int = Query(1, ge=1),
page_size: int = Query(20, ge=1, le=100),
action: str | None = Query(None, description="Filter by action"),
resource_type: str | None = Query(None),
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""List recent audit events (newest first). Requires audit:read."""
query = select(AuditLog)
count_query = select(func.count(AuditLog.id))
if action:
query = query.where(AuditLog.action == action)
count_query = count_query.where(AuditLog.action == action)
if resource_type:
query = query.where(AuditLog.resource_type == resource_type)
count_query = count_query.where(AuditLog.resource_type == resource_type)
total = (await session.execute(count_query)).scalar() or 0
result = await session.execute(
query.order_by(AuditLog.created_at.desc())
.offset((page - 1) * page_size)
.limit(page_size)
)
rows = result.scalars().all()
items = [
{
"id": r.id,
"actor": r.actor,
```

```
"action": r.action,
"resource_type": r.resource_type,
"resource_id": r.resource_id,
"detail": r.detail,
"request_id": r.request_id,
"ip_address": r.ip_address,
"created_at": r.created_at.isoformat() if r.created_at else None,
}
for r in rows
]
return {"items": items, "total": total, "page": page, "page_size": page_size}
# === File: app/api/v1/endpoints/benchmarks.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Benchmark platform API."""
from __future__ import annotations
from typing import Any
from fastapi import APIRouter, Depends, File, Query, Request, UploadFile
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.benchmark.service import get_benchmark_service
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.core.tenant_context import (
current_tenant_id,
extract_tenant_header,
resolve_tenant_context,
)
from app.utils.exceptions import AgentFlowError
router = APIRouter()
def _actor(request: Request) -> str:
return getattr(request.state, "actor", None) or "anonymous"
async def _bind_tenant(request: Request, session: AsyncSession) -> str | None:
role = getattr(request.state, "role", None)
role_s = role.value if hasattr(role, "value") else str(role or "")
await resolve_tenant_context(
session,
actor=_actor(request),
header_value=extract_tenant_header(request),
system_role=role_s,
)
return current_tenant_id()
class BenchmarkCreate(BaseModel):
name: str = Field(..., min_length=1, max_length=255)
description: str = ""
tags: list[str] = Field(default_factory=list)
cases: list[dict[str, Any]] = Field(default_factory=list)
version: str = Field(default="1.0", max_length=64)
scorecard: dict[str, Any] | None = Field(
default=None, description="Optional Scorecard JSON (Phase 3)"
```

```
)
source_task_id: str | None = Field(
default=None, description="If set, clone suites from this Task as cases"
)
class BenchmarkRunBody(BaseModel):
label: str = "default"
agent_config: dict[str, Any] = Field(
default_factory=lambda: {
"runner": "openai",
"model": "gpt-4o-mini",
"temperature": 0,
}
)
enqueue: bool = True
class ImportBody(BaseModel):
format: str = Field("json", description="json | csv")
content: str = Field(..., min_length=1)
class CompareRunsBody(BaseModel):
"""Compare current run against a baseline (defaults to previous completed run)."""
current_run_id: str
baseline_run_id: str | None = Field(
default=None,
description="If omitted, use the previous completed run before current",
)
score_stable_eps: float = Field(
default=1.0, ge=0, description="|Δscore| below this → stable"
)
def _bm_dict(bm, *, case_count: int | None = None) -> dict[str, Any]:
meta = dict(bm.meta or {})
return {
"id": bm.id,
"name": bm.name,
"description": bm.description,
"status": bm.status,
"created_by": bm.created_by,
"tags": bm.tags or [],
"tenant_id": bm.tenant_id,
"version": meta.get("version") or "1.0",
"scorecard": meta.get("scorecard"),
"source_task_id": meta.get("source_task_id"),
"case_count": case_count
if case_count is not None
else (len(bm.cases) if getattr(bm, "cases", None) is not None else None),
"created_at": bm.created_at.isoformat() if bm.created_at else None,
}
def _run_dict(run) -> dict[str, Any]:
return {
"id": run.id,
"benchmark_id": run.benchmark_id,
"task_id": run.task_id,
```

```
"label": run.label,
"status": run.status,
"agent_config": run.agent_config or {},
"summary": run.summary or {},
"created_by": run.created_by,
"created_at": run.created_at.isoformat() if run.created_at else None,
"started_at": run.started_at.isoformat() if run.started_at else None,
"finished_at": run.finished_at.isoformat() if run.finished_at else None,
}
@router.get("")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def list_benchmarks(
request: Request,
limit: int = Query(50, ge=1, le=200),
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
tenant_id = await _bind_tenant(request, session)
svc = get_benchmark_service()
items = await svc.list_benchmarks(
session, actor=_actor(request), tenant_id=tenant_id, limit=limit
)
out = []
for bm in items:
full = await svc.get_benchmark(session, bm.id, with_cases=True)
out.append(_bm_dict(full, case_count=len(full.cases)))
return {"items": out, "total": len(out)}
@router.post("", status_code=201)
@require_permission(
Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def create_benchmark(
body: BenchmarkCreate,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
tenant_id = await _bind_tenant(request, session)
svc = get_benchmark_service()
if body.source_task_id:
bm = await svc.create_from_task(
session,
task_id=body.source_task_id,
name=body.name,
description=body.description,
version=body.version,
created_by=_actor(request),
tenant_id=tenant_id,
scorecard=body.scorecard,
)
if body.cases:
await svc.add_cases(
```

```
session, bm, body.cases, tenant_id=tenant_id
)
else:
bm = await svc.create_benchmark(
session,
name=body.name,
description=body.description,
created_by=_actor(request),
tenant_id=tenant_id,
tags=body.tags,
cases=body.cases,
version=body.version,
scorecard=body.scorecard,
)
await session.commit()
full = await svc.get_benchmark(session, bm.id, with_cases=True)
return {
**_bm_dict(full, case_count=len(full.cases)),
"cases": [
{
"id": c.id,
"name": c.name,
"user_query": c.user_query,
"expected_output": c.expected_output,
"expected_tools": c.expected_tools,
"weight": c.weight,
}
for c in full.cases
],
}
@router.get("/{benchmark_id}")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def get_benchmark(
benchmark_id: str,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
await _bind_tenant(request, session)
svc = get_benchmark_service()
bm = await svc.get_benchmark(session, benchmark_id, with_cases=True)
return {
**_bm_dict(bm, case_count=len(bm.cases)),
"cases": [
{
"id": c.id,
"name": c.name,
"user_query": c.user_query,
"expected_output": c.expected_output,
"expected_tools": c.expected_tools,
"weight": c.weight,
```

```
}
for c in bm.cases
],
}
@router.post("/{benchmark_id}/import")
@require_permission(
Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def import_cases(
benchmark_id: str,
body: ImportBody,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
tenant_id = await _bind_tenant(request, session)
svc = get_benchmark_service()
bm = await svc.get_benchmark(session, benchmark_id, with_cases=False)
try:
cases = svc.parse_import_payload(content=body.content, fmt=body.format)
except AgentFlowError:
raise
except Exception as exc:
raise AgentFlowError(f"import parse failed: {exc}", status_code=422) from exc
created = await svc.add_cases(session, bm, cases, tenant_id=tenant_id)
await session.commit()
return {"imported": len(created), "benchmark_id": bm.id}
@router.post("/{benchmark_id}/import/file")
@require_permission(
Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def import_cases_file(
benchmark_id: str,
request: Request,
file: UploadFile = File(...),
format: str = Query("json", description="json | csv"),
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
tenant_id = await _bind_tenant(request, session)
raw = await file.read()
content = raw.decode("utf-8", errors="replace")
name = (file.filename or "").lower()
fmt = format
if name.endswith(".csv"):
fmt = "csv"
elif name.endswith(".json"):
fmt = "json"
svc = get_benchmark_service()
bm = await svc.get_benchmark(session, benchmark_id, with_cases=False)
cases = svc.parse_import_payload(content=content, fmt=fmt)
created = await svc.add_cases(session, bm, cases, tenant_id=tenant_id)
```

```
await session.commit()
return {"imported": len(created), "benchmark_id": bm.id, "format": fmt}
@router.post("/{benchmark_id}/run", status_code=201)
@require_permission(
Permission.BENCHMARK_CREATE, Permission.TASK_EXECUTE, require_all=False
)
async def run_benchmark(
benchmark_id: str,
body: BenchmarkRunBody,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Run benchmark through Evaluation Engine (creates Task + enqueues)."""
tenant_id = await _bind_tenant(request, session)
svc = get_benchmark_service()
try:
run = await svc.run_benchmark(
session,
benchmark_id=benchmark_id,
actor=_actor(request),
agent_config=body.agent_config,
label=body.label,
tenant_id=tenant_id,
enqueue=body.enqueue,
)
except AgentFlowError:
raise
await session.commit()
return {"run": _run_dict(run)}
@router.get("/{benchmark_id}/runs")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def list_runs(
benchmark_id: str,
request: Request,
limit: int = Query(50, ge=1, le=200),
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""History of regression evaluation runs for continuous evaluation."""
await _bind_tenant(request, session)
svc = get_benchmark_service()
runs = await svc.list_runs(session, benchmark_id, limit=limit)
await session.commit()
return {
"benchmark_id": benchmark_id,
"items": [_run_dict(r) for r in runs],
"total": len(runs),
}
@router.post("/{benchmark_id}/runs/{run_id}/finalize")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def finalize_run(
```

```
benchmark_id: str,
run_id: str,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
await _bind_tenant(request, session)
svc = get_benchmark_service()
run = await svc.finalize_run(session, run_id)
if run.benchmark_id != benchmark_id:
raise AgentFlowError("Run does not belong to benchmark", status_code=400)
await session.commit()
return {"run": _run_dict(run)}
@router.post("/{benchmark_id}/compare")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def compare_runs(
benchmark_id: str,
body: CompareRunsBody,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Compare two runs (or current vs previous) for regression detection."""
await _bind_tenant(request, session)
svc = get_benchmark_service()
await svc.get_benchmark(session, benchmark_id, with_cases=False)
current = await svc.get_run(session, body.current_run_id)
if current.benchmark_id != benchmark_id:
raise AgentFlowError("current_run_id not in this benchmark", status_code=400)
if body.baseline_run_id:
baseline = await svc.get_run(session, body.baseline_run_id)
if baseline.benchmark_id != benchmark_id:
raise AgentFlowError(
"baseline_run_id not in this benchmark", status_code=400
)
else:
history = await svc.list_runs(session, benchmark_id, limit=100)
baseline = None
for r in history:
if r.id == current.id:
continue
if r.created_at and current.created_at and r.created_at >= current.created_at:
continue
if r.status == "completed" or (r.summary or {}).get("score") is not None:
baseline = r
break
if baseline is None:
raise AgentFlowError(
"No baseline run found — provide baseline_run_id or complete a prior run",
status_code=400,
)
result = svc.compare_runs(
```

```
current, baseline, score_stable_eps=body.score_stable_eps
)
await session.commit()
return {"benchmark_id": benchmark_id, **result}
@router.get("/{benchmark_id}/leaderboard")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def leaderboard(
benchmark_id: str,
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
await _bind_tenant(request, session)
svc = get_benchmark_service()
await svc.get_benchmark(session, benchmark_id, with_cases=False)
board = await svc.leaderboard(session, benchmark_id)
await session.commit()
return {
"benchmark_id": benchmark_id,
"items": board,
"total": len(board),
"metrics": [
"score",
"success_rate",
"score_coverage",
"accuracy",
"quality",
"latency_ms",
"cost",
"tokens",
],
}
# === File: app/api/v1/endpoints/billing.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Billing / subscription / usage APIs (BILLING_ENABLED optional)."""
from __future__ import annotations
import json
from typing import Any
from fastapi import APIRouter, Depends, Query, Request # Query used by rollover
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.billing.service import billing_enabled, get_billing_service
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.utils.exceptions import AgentFlowError
router = APIRouter()
def _actor(request: Request) -> str:
return getattr(request.state, "actor", None) or "anonymous"
class SubscribeBody(BaseModel):
plan_code: str = Field(..., description="free | pro | enterprise")
```

```
def _plan_dict(p) -> dict[str, Any]:
return {
"id": p.id,
"code": p.code,
"name": p.name,
"description": p.description,
"price_month_cents": p.price_month_cents,
"billing_cycle": getattr(p, "billing_cycle", None) or "monthly",
"token_quota": p.token_quota,
"task_quota": p.task_quota,
"storage_quota_mb": getattr(p, "storage_quota_mb", None),
"plugin_quota": getattr(p, "plugin_quota", None),
"features": p.features or {},
"limits": (p.features or {}).get("limits")
or {
"tasks": p.task_quota,
"tokens": p.token_quota,
"storage_mb": getattr(p, "storage_quota_mb", None),
"plugins": getattr(p, "plugin_quota", None),
},
"is_public": p.is_public,
}
@router.get("/plans")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def list_plans(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
svc = get_billing_service()
await svc.ensure_default_plans(session)
plans = await svc.list_plans(session, public_only=True)
return {
"items": [_plan_dict(p) for p in plans],
"total": len(plans),
"billing_enabled": billing_enabled(),
}
@router.get("/plan")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def get_current_plan(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Current actor plan + quota (enterprise contract alias)."""
actor = _actor(request)
svc = get_billing_service()
return await svc.get_current_plan(session, actor)
@router.get("/quota")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def get_quota(
request: Request,
```

```
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
actor = _actor(request)
svc = get_billing_service()
return await svc.get_quota(session, actor)
@router.get("/usage")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def list_usage(
request: Request,
limit: int = Query(50, ge=1, le=500),
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
actor = _actor(request)
svc = get_billing_service()
rows = await svc.list_usage(session, actor, limit=limit)
return {
"items": [
{
"id": r.id,
"metric": r.metric,
"quantity": r.quantity,
"ref_type": r.ref_type,
"ref_id": r.ref_id,
"trace_id": r.trace_id,
"created_at": r.created_at.isoformat() if r.created_at else None,
}
for r in rows
],
"total": len(rows),
"billing_enabled": billing_enabled(),
}
class CheckoutBody(BaseModel):
plan_code: str = Field(..., description="pro | enterprise (not free)")
success_url: str | None = None
cancel_url: str | None = None
class MockConfirmBody(BaseModel):
session_id: str
plan_code: str
actor: str | None = None
@router.post("/subscribe")
@require_permission(
Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def subscribe(
request: Request,
body: SubscribeBody,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Direct subscribe (ops / free tier / mock without payment).
Paid plans in production should use ``POST /billing/checkout``.
```

```
"""
actor = _actor(request)
svc = get_billing_service()
await svc.ensure_default_plans(session)
try:
sub = await svc.subscribe(session, actor=actor, plan_code=body.plan_code)
except AgentFlowError:
raise
except Exception as exc:
raise AgentFlowError(f"subscribe failed: {exc}", status_code=400) from exc
await session.commit()
return {
"subscription": {
"id": sub.id,
"actor": sub.actor,
"plan_id": sub.plan_id,
"status": sub.status,
}
}
@router.post("/checkout")
@require_permission(
Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def create_checkout(
request: Request,
body: CheckoutBody,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Create Stripe Checkout session (mock by default).
Returns ``url`` to redirect the browser. In mock mode, complete with
``POST /billing/checkout/mock-confirm``.
"""
from app.core.billing.stripe_checkout import create_checkout_session, stripe_mode
actor = _actor(request)
svc = get_billing_service()
await svc.ensure_default_plans(session)
plan = await svc.get_plan_by_code(session, body.plan_code)
if plan is None:
raise AgentFlowError(f"Unknown plan: {body.plan_code}", status_code=404)
if plan.code == "free" or plan.price_month_cents <= 0:
sub = await svc.subscribe(session, actor=actor, plan_code="free")
await session.commit()
return {
"mode": "direct",
"url": None,
"subscription": {
"id": sub.id,
"actor": sub.actor,
"plan_id": sub.plan_id,
"status": sub.status,
```

```
},
}
try:
session_out = create_checkout_session(
actor=actor,
plan_code=plan.code,
plan_name=plan.name,
amount_cents=plan.price_month_cents,
success_url=body.success_url,
cancel_url=body.cancel_url,
)
except Exception as exc:
raise AgentFlowError(f"checkout failed: {exc}", status_code=400) from exc
return {
"checkout": session_out,
"stripe_mode": stripe_mode(),
"billing_enabled": billing_enabled(),
}
@router.post("/checkout/mock-confirm")
@require_permission(
Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def mock_confirm_checkout(
request: Request,
body: MockConfirmBody,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Complete a mock Checkout session and activate the subscription."""
from app.core.billing.stripe_checkout import (
build_mock_completed_event,
parse_checkout_completed_event,
stripe_mode,
)
if stripe_mode() != "mock":
raise AgentFlowError(
"mock-confirm only available when STRIPE_MODE=mock",
status_code=400,
)
actor = body.actor or _actor(request)
event = build_mock_completed_event(
actor=actor,
plan_code=body.plan_code,
session_id=body.session_id,
)
parsed = parse_checkout_completed_event(event)
if not parsed:
raise AgentFlowError("invalid mock session", status_code=400)
svc = get_billing_service()
await svc.ensure_default_plans(session)
try:
```

```
sub = await svc.subscribe(
session, actor=parsed["actor"], plan_code=parsed["plan_code"]
)
sub.external_ref = parsed.get("external_ref") or body.session_id
except Exception as exc:
raise AgentFlowError(f"activate failed: {exc}", status_code=400) from exc
await session.commit()
return {
"ok": True,
"mode": "mock",
"subscription": {
"id": sub.id,
"actor": sub.actor,
"plan_id": sub.plan_id,
"status": sub.status,
"external_ref": sub.external_ref,
},
}
async def _handle_stripe_webhook(
request: Request,
session: AsyncSession,
) -> dict[str, Any]:
"""Shared Stripe/mock webhook body (public — signature verified)."""
from app.core.billing.stripe_checkout import (
parse_checkout_completed_event,
verify_webhook_signature,
)
payload = await request.body()
sig = request.headers.get("stripe-signature") or request.headers.get(
"Stripe-Signature"
)
if not verify_webhook_signature(payload, sig):
raise AgentFlowError("invalid webhook signature", status_code=400)
try:
event = json.loads(payload.decode("utf-8") or "{}")
except Exception as exc:
raise AgentFlowError(f"invalid JSON: {exc}", status_code=400) from exc
parsed = parse_checkout_completed_event(event)
if not parsed:
return {"received": True, "handled": False, "reason": "ignored_event"}
svc = get_billing_service()
await svc.ensure_default_plans(session)
try:
sub = await svc.subscribe(
session,
actor=parsed["actor"],
plan_code=parsed["plan_code"],
)
if parsed.get("external_ref"):
sub.external_ref = parsed["external_ref"]
```

```
await session.commit()
except Exception as exc:
await session.rollback()
raise AgentFlowError(
f"webhook activate failed: {exc}", status_code=400
) from exc
try:
from app.core.audit import write_audit
await write_audit(
session,
action="billing.checkout_completed",
resource_type="subscription",
resource_id=sub.id,
actor=parsed["actor"],
detail={
"plan_code": parsed["plan_code"],
"session_id": parsed.get("session_id"),
"source": "stripe_webhook",
},
)
await session.commit()
except Exception:
pass
return {
"received": True,
"handled": True,
"actor": parsed["actor"],
"plan_code": parsed["plan_code"],
"subscription_id": sub.id,
}
@router.post("/webhook/stripe")
async def stripe_webhook(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Stripe webhook receiver (public path — signature verified)."""
return await _handle_stripe_webhook(request, session)
@router.post("/webhook")
async def billing_webhook(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Stripe-compatible webhook alias (POST /api/v1/billing/webhook)."""
return await _handle_stripe_webhook(request, session)
@router.get("/invoices")
@require_permission(Permission.TASK_READ)
async def list_invoices(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
```

```
actor = _actor(request)
svc = get_billing_service()
invs = await svc.list_invoices(session, actor)
return {
"items": [
{
"id": i.id,
"period": i.period,
"amount_cents": i.amount_cents,
"status": i.status,
"line_items": i.line_items,
"issued_at": i.issued_at.isoformat() if i.issued_at else None,
}
for i in invs
],
"total": len(invs),
}
@router.post("/invoices/draft")
@require_permission(Permission.SYSTEM_CONFIG)
async def create_draft_invoice(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
actor = _actor(request)
svc = get_billing_service()
inv = await svc.draft_invoice(session, actor)
await session.commit()
return {
"invoice": {
"id": inv.id,
"period": inv.period,
"amount_cents": inv.amount_cents,
"status": inv.status,
"line_items": inv.line_items,
}
}
@router.post("/quota/rollover")
@require_permission(Permission.SYSTEM_CONFIG)
async def rollover_quota(
request: Request,
session: AsyncSession = Depends(get_db),
all_actors: bool = Query(False, description="Reset all subscribed actors"),
) -> dict[str, Any]:
"""Create a fresh quota balance for the current calendar month.
Idempotent for the current period. Used by ops cron or manual admin.
"""
svc = get_billing_service()
await svc.ensure_default_plans(session)
if all_actors:
n = await svc.rollover_all_active(session)
```

```
await session.commit()
return {"rolled": n, "scope": "all_active"}
actor = _actor(request)
bal = await svc.rollover_period(session, actor)
await session.commit()
return {
"rolled": 1,
"scope": "self",
"quota": {
"actor": bal.actor,
"period": bal.period,
"task_used": bal.task_used,
"task_limit": bal.task_limit,
"token_used": bal.token_used,
"token_limit": bal.token_limit,
},
}
# === File: app/api/v1/endpoints/dashboard.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Dashboard stats API with short-TTL cache + Intelligence Center overview."""
from __future__ import annotations
from typing import Any
from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.dependencies import get_db
from app.core.rbac import Permission, get_request_role, require_permission
router = APIRouter()
@router.get("/stats")
@require_permission(Permission.TASK_READ)
async def dashboard_stats(
request: Request,
session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
"""Return aggregated task counts for the current actor (cached 1 min)."""
from app.core.cache.services import get_cached_dashboard
actor = getattr(request.state, "actor", None) or "anonymous"
role = get_request_role(request).value
return await get_cached_dashboard(session, actor=actor, role=role)
def _build_topology(
*,
running: int,
completed: int,
failed: int,
avg_score: float | None,
latency_ms: float | None,
tokens: int,
success_rate: float | None,
model_hint: str | None = None,
) -> dict[str, Any]:
```

```
"""Horizontal ReAct pipeline topology for ReactFlow cockpit."""
tool_status = (
"error" if failed > max(completed, 1) else ("warn" if failed else "ok")
)
planner_status = "ok" if running or completed else "idle"
if running:
planner_status = "ok"
judge_status = "ok" if completed else ("warn" if failed else "idle")
observe_status = "warn" if failed else "ok"
lat_label = f"{round(latency_ms)} ms" if latency_ms is not None else "—"
score_label = f"{avg_score:.1f}" if avg_score is not None else "—"
sr = f"{success_rate * 100:.1f}%" if success_rate is not None else "—"
nodes = [
{
"id": "user",
"label": "User Request",
"status": "ok",
"kind": "ingress",
"meta": {
"type": "ingress",
"input": "Business query / test suite",
"output": "→ Planner",
"latency": "< 1 ms",
"model": "—",
},
},
{
"id": "planner",
"label": "Planner Agent",
"status": planner_status,
"kind": "agent",
"meta": {
"type": "agent",
"running": running,
"input": "User query + system prompt",
"output": "Thought / plan / tool plan",
"token": tokens // max(running + completed, 1) if tokens else 0,
"latency": lat_label,
"model": model_hint or "openai-compatible",
},
},
{
"id": "tool",
"label": "Tool Calling",
"status": tool_status,
"kind": "tool",
"meta": {
"type": "tool",
"failed": failed,
"input": "Function call args",
```

```
"output": "Tool observation payload",
"latency": "sandbox",
"error": f"{failed} failed tasks" if failed else "",
},
},
{
"id": "observe",
"label": "Observation",
"status": observe_status,
"kind": "observe",
"meta": {
"type": "observe",
"input": "Tool result",
"output": "Normalized observation",
"latency": lat_label,
},
},
{
"id": "judge",
"label": "LLM Judge",
"status": judge_status,
"kind": "judge",
"meta": {
"type": "judge",
"score": avg_score,
"input": "Trace + expected",
"output": f"Score {score_label} • SR {sr}",
"latency": lat_label,
"model": "judge-engine",
},
},
]
edges = [
{"source": "user", "target": "planner", "label": "dispatch"},
{"source": "planner", "target": "tool", "label": "act"},
{"source": "tool", "target": "observe", "label": "result"},
{"source": "observe", "target": "judge", "label": "score"},
]
if failed:
edges.append(
{
"source": "observe",
"target": "planner",
"type": "loop",
"label": "retry",
}
)
return {
"layout": "horizontal",
"nodes": nodes,
```

—— 前 30 页之后省略中间部分源代码 ——

（前30页完，以下为后30页）

```
    item = self._judges.get(str(key).lower().strip())
    return item[1] if item else None

def list_judges(self) -> list[dict[str, Any]]:
    return [
        {"key": k, "plugin_id": pid} for k, (pid, _) in sorted(self._judges.items())
    ]

# ---- tools ----
def register_tool(self, spec: ToolSpec, *, plugin_id: str) -> None:
    if not spec.name:
        raise ValueError("tool name required")
    if not callable(spec.fn):
        raise ValueError(f"tool {spec.name!r} requires callable fn")
    spec.source_plugin = plugin_id
    self._tools[spec.name] = spec
    logger.info("Registered plugin tool %r from %s", spec.name, plugin_id)

def unregister_tools(self, plugin_id: str) -> int:
    keys = [k for k, s in self._tools.items() if s.source_plugin == plugin_id]
    for k in keys:
        del self._tools[k]
    return len(keys)

def get_tool(self, name: str) -> ToolSpec | None:
    return self._tools.get(name)

def get_tool_fn(self, name: str) -> Callable[..., str] | None:
    spec = self._tools.get(name)
    return spec.fn if spec else None

def list_tools(self) -> list[dict[str, Any]]:
    out = []
    for name, spec in sorted(self._tools.items()):
        out.append(
            {
                "name": name,
                "description": spec.description,
                "parameters": spec.parameters,
                "required": list(spec.required),
                "network": spec.network,
                "plugin_id": spec.source_plugin,
            }
        )
    return out

def clear(self) -> None:
    self._runners.clear()
    self._judges.clear()
    self._tools.clear()
```

```
_default_caps: PluginCapabilityRegistry | None = None


def get_capability_registry() -> PluginCapabilityRegistry:
    global _default_caps
    if _default_caps is None:
        _default_caps = PluginCapabilityRegistry()
    return _default_caps


def reset_capability_registry() -> PluginCapabilityRegistry:
    global _default_caps
    _default_caps = PluginCapabilityRegistry()
    return _default_caps
# === File: app/core/plugins/sandbox.py ===
  # AgentFlow-Eval V1.0.0
"""Plugin sandbox declarations — permission & resource constraints."""

from __future__ import annotations

from dataclasses import asdict, dataclass, field
from typing import Any


@dataclass
class PluginSandboxPolicy:
    """Declarative isolation policy (enforced at activate time)."""

    allow_network: bool = False
    max_cpu_ms: int = 30_000
    max_memory_mb: int = 512
    # Required host permissions (RBAC strings) to activate
    permissions: list[str] = field(default_factory=list)
    # Filesystem roots allowed (empty = none beyond in-memory)
    allowed_paths: list[str] = field(default_factory=list)

    def to_dict(self) -> dict[str, Any]:
        return asdict(self)

    @classmethod
    def from_mapping(cls, data: dict[str, Any] | None) -> "PluginSandboxPolicy":
        data = data or {}
        return cls(
            allow_network=bool(data.get("allow_network", False)),
            max_cpu_ms=int(data.get("max_cpu_ms") or 30_000),
            max_memory_mb=int(data.get("max_memory_mb") or 512),
            permissions=list(data.get("permissions") or []),
            allowed_paths=list(data.get("allowed_paths") or []),
```

```
        )


def validate_activate(
    policy: PluginSandboxPolicy,
    *,
    actor_permissions: set[str] | None = None,
    rbac_enforced: bool = False,
) -> tuple[bool, str]:
    """Check whether caller may activate plugin under policy."""
    if not rbac_enforced:
        return True, "rbac off"
    needed = set(policy.permissions or [])
    if not needed:
        return True, "no extra permissions"
    have = actor_permissions or set()
    missing = needed - have
    if missing:
        return False, f"missing permissions: {sorted(missing)}"
    return True, "ok"
# === File: app/core/plugins/signature.py ===
  # AgentFlow-Eval V1.0.0
"""Optional plugin integrity / signature checks for production hardening.

When ``PLUGIN_SIGNATURE_CHECK=true``, every loadable plugin module path must
have a matching HMAC-SHA256 digest listed in ``PLUGIN_SIGNATURES``.

Format of PLUGIN_SIGNATURES (comma-separated)::

    module.path:hexdigest,other.mod:hexdigest

Digest = HMAC-SHA256(PLUGIN_SIGNING_SECRET, utf-8 file bytes).hexdigest()
"""

from __future__ import annotations

import hashlib
import hmac
import importlib.util
import logging
from pathlib import Path
from typing import Any

from app.config import settings

logger = logging.getLogger(__name__)


def signature_check_enabled() -> bool:
    return bool(getattr(settings, "PLUGIN_SIGNATURE_CHECK", False))
```

```
def parse_signature_map(raw: str | None = None) -> dict[str, str]:
    text = raw if raw is not None else getattr(settings, "PLUGIN_SIGNATURES", "") or ""
    out: dict[str, str] = {}
    for part in str(text).split(","):
        part = part.strip()
        if not part or ":" not in part:
            continue
        mod, dig = part.rsplit(":", 1)
        mod, dig = mod.strip(), dig.strip().lower()
        if mod and dig:
            out[mod] = dig
    return out


def compute_file_hmac(path: Path, secret: str) -> str:
    data = path.read_bytes()
    return hmac.new(secret.encode("utf-8"), data, hashlib.sha256).hexdigest()


def resolve_module_file(module_entry: str) -> Path | None:
    """Resolve ``pkg.mod:Class`` or ``pkg.mod`` to a filesystem path."""
    mod_path = module_entry.split(":", 1)[0].strip()
    if not mod_path:
        return None
    try:
        spec = importlib.util.find_spec(mod_path)
    except (ModuleNotFoundError, ValueError):
        return None
    if spec is None or not spec.origin or spec.origin == "built-in":
        return None
    p = Path(spec.origin)
    return p if p.is_file() else None


def verify_plugin_entry(module_entry: str) -> tuple[bool, str]:
    """Return (ok, reason). When check disabled → always ok."""
    if not signature_check_enabled():
        return True, "disabled"
    secret = (getattr(settings, "PLUGIN_SIGNING_SECRET", None) or "").strip()
    if not secret:
        return False, "PLUGIN_SIGNING_SECRET missing while PLUGIN_SIGNATURE_CHECK=true"
    sigs = parse_signature_map()
    mod = module_entry.split(":", 1)[0].strip()
    expected = sigs.get(mod) or sigs.get(module_entry)
    if not expected:
        return False, f"no signature registered for {mod}"
    path = resolve_module_file(module_entry)
    if path is None:
```

```
        # Allow pure package path failures only if explicit digest maps to entry
        return False, f"cannot resolve module file for {mod}"
    actual = compute_file_hmac(path, secret)
    if not hmac.compare_digest(actual, expected):
        return False, f"signature mismatch for {mod}"
    return True, "ok"


def filter_signed_modules(modules: list[str]) -> tuple[list[str], list[dict[str, Any]]]:
    """Filter module entries; return (allowed, rejections)."""
    if not signature_check_enabled():
        return list(modules), []
    allowed: list[str] = []
    rejected: list[dict[str, Any]] = []
    for m in modules:
        ok, reason = verify_plugin_entry(m)
        if ok:
            allowed.append(m)
        else:
            logger.warning("Plugin signature rejected %s: %s", m, reason)
            rejected.append({"module": m, "reason": reason})
    return allowed, rejected
# === File: app/core/plugins/versioning.py ===
  # AgentFlow-Eval V1.0.0
"""Plugin semver helpers and core compatibility checks."""

from __future__ import annotations

import re
from typing import Any

_CORE_VERSION = "0.1.0"
_SEMVER = re.compile(
    r"^(?P<major>0|[1-9]\d*)\.(?P<minor>0|[1-9]\d*)\.(?P<patch>0|[1-9]\d*)"
    r"(?:-(?P<pre>[0-9A-Za-z.-]+))?(?:\+(?P<build>[0-9A-Za-z.-]+))?$"
)


def parse_semver(version: str) -> tuple[int, int, int]:
    m = _SEMVER.match((version or "").strip())
    if not m:
        return (0, 0, 0)
    return int(m.group("major")), int(m.group("minor")), int(m.group("patch"))


def cmp_semver(a: str, b: str) -> int:
    ta, tb = parse_semver(a), parse_semver(b)
    return (ta > tb) - (ta < tb)
```

```
def satisfies(version: str, requirement: str) -> bool:
    """Minimal range: ``>=x.y.z``, ``==x.y.z``, ``*`` / empty = any."""
    req = (requirement or "").strip()
    if not req or req == "*":
        return True
    if req.startswith(">="):
        return cmp_semver(version, req[2:].strip()) >= 0
    if req.startswith("=="):
        return cmp_semver(version, req[2:].strip()) == 0
    if req.startswith(">"):
        return cmp_semver(version, req[1:].strip()) > 0
    return cmp_semver(version, req) >= 0


def check_core_requirement(requires_core: str | None) -> tuple[bool, str]:
    """Return (ok, message) for plugin requires_core field."""
    if not requires_core:
        return True, "ok"
    ok = satisfies(_CORE_VERSION, requires_core)
    if ok:
        return True, f"core {_CORE_VERSION} satisfies {requires_core}"
    return False, f"core {_CORE_VERSION} does not satisfy requires_core={requires_core}"


def plugin_version_info(meta: dict[str, Any] | None) -> dict[str, Any]:
    meta = meta or {}
    ver = str(meta.get("version") or "0.1.0")
    requires = meta.get("requires_core") or meta.get("extra", {}).get("requires_core")
    ok, msg = check_core_requirement(str(requires) if requires else None)
    return {
        "version": ver,
        "requires_core": requires,
        "compatible": ok,
        "message": msg,
        "core_version": _CORE_VERSION,
    }
# === File: app/core/ports/__init__.py ===
  # AgentFlow-Eval V1.0.0
"""Infrastructure ports — business code depends on these, not on Redis/Celery."""

from app.core.ports.task_queue import TaskQueuePort, EnqueueResult
from app.core.ports.cache import CachePort
from app.core.ports.event_bus import EventBusPort
from app.core.ports.metering import MeteringPort

__all__ = [
    "TaskQueuePort",
    "EnqueueResult",
    "CachePort",
    "EventBusPort",
```

```
    "MeteringPort",
]
# === File: app/core/ports/cache.py ===
  # AgentFlow-Eval V1.0.0
"""Cache port — L1/L2 or memory-only depending on deploy profile."""

from __future__ import annotations

from typing import Any, Protocol, runtime_checkable


@runtime_checkable
class CachePort(Protocol):
    """Minimal async cache interface used by application services."""

    @property
    def backend_name(self) -> str: ...

    async def get(self, key: str) -> Any | None: ...

    async def set(self, key: str, value: Any, ttl: int | None = None) -> None: ...

    async def delete(self, key: str) -> None: ...
# === File: app/core/ports/event_bus.py ===
  # AgentFlow-Eval V1.0.0
"""Event bus port — Redis pub/sub or in-process fan-out."""

from __future__ import annotations

from typing import Any, Protocol, runtime_checkable


@runtime_checkable
class EventBusPort(Protocol):
    """Publish domain/activity events across processes (or in-process for lite)."""

    @property
    def backend_name(self) -> str: ...

    def publish(self, channel: str, payload: dict[str, Any]) -> None:
        """Sync-safe publish (Celery workers + FastAPI)."""
        ...
# === File: app/core/ports/metering.py ===
  # AgentFlow-Eval V1.0.0
"""Metering port — usage / billing hooks (noop until BILLING_ENABLED)."""

from __future__ import annotations

from typing import Any, Protocol, runtime_checkable
```

```
@runtime_checkable
class MeteringPort(Protocol):
    """Record consumable usage (tokens, tasks, judge calls)."""

    @property
    def backend_name(self) -> str: ...

    def record(
        self,
        *,
        actor: str,
        metric: str,
        quantity: float = 1.0,
        ref_type: str | None = None,
        ref_id: str | None = None,
        extra: dict[str, Any] | None = None,
    ) -> None:
        """Best-effort record; must never raise into the business path."""
        ...
# === File: app/core/ports/task_queue.py ===
  # AgentFlow-Eval V1.0.0
"""Task queue port — abstract async job dispatch (Celery / Eager / Memory / Arq)."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Protocol, runtime_checkable


@dataclass(frozen=True)
class EnqueueResult:
    """Result of enqueueing a background job."""

    task_id: str
    backend: str
    eager: bool = False


@runtime_checkable
class TaskQueuePort(Protocol):
    """Pluggable job queue.

    ``name`` is a logical task name registered by the active adapter
    (e.g. ``run_full_evaluation``).
    """

    @property
    def backend_name(self) -> str:
        """Adapter identifier: celery | eager | memory | arq."""
```

```
        ...

    def is_eager(self) -> bool:
        """True when jobs run in-process (no broker required)."""
        ...

    def enqueue(
        self,
        name: str,
        *,
        args: tuple[Any, ...] = (),
        kwargs: dict[str, Any] | None = None,
        countdown: float = 0.0,
    ) -> EnqueueResult:
        """Submit a job; returns opaque task id for revoke/tracking."""
        ...

    def revoke(self, task_id: str, *, terminate: bool = True) -> None:
        """Best-effort cancel of a running/queued job."""
        ...
# === File: app/core/profiles/__init__.py ===
  # AgentFlow-Eval V1.0.0
"""Deploy profiles — bind ports/adapters without rewriting business logic."""

from __future__ import annotations

import logging
from typing import Any, Literal

from app.core.ports.cache import CachePort
from app.core.ports.event_bus import EventBusPort
from app.core.ports.metering import MeteringPort
from app.core.ports.task_queue import TaskQueuePort

logger = logging.getLogger(__name__)

ProfileName = Literal["lite", "private", "saas"]

_queue: TaskQueuePort | None = None
_cache: CachePort | None = None
_bus: EventBusPort | None = None
_meter: MeteringPort | None = None
_profile: str = "private"
_applied: bool = False


def get_task_queue() -> TaskQueuePort:
    global _queue
    if _queue is None:
        apply_profile_from_settings()
```

```
    assert _queue is not None
    return _queue


def get_cache_port() -> CachePort:
    global _cache
    if _cache is None:
        apply_profile_from_settings()
    assert _cache is not None
    return _cache


def get_event_bus() -> EventBusPort:
    global _bus
    if _bus is None:
        apply_profile_from_settings()
    assert _bus is not None
    return _bus


def get_meter() -> MeteringPort:
    global _meter
    if _meter is None:
        apply_profile_from_settings()
    assert _meter is not None
    return _meter


def current_profile() -> str:
    return _profile


def bind_ports(
    *,
    queue: TaskQueuePort,
    cache: CachePort,
    bus: EventBusPort,
    meter: MeteringPort,
    profile: str,
) -> None:
    global _queue, _cache, _bus, _meter, _profile, _applied
    _queue = queue
    _cache = cache
    _bus = bus
    _meter = meter
    _profile = profile
    _applied = True


def reset_ports() -> None:
```

```
    """Test helper — clear bindings so next get_* re-applies profile."""
    global _queue, _cache, _bus, _meter, _applied
    _queue = None
    _cache = None
    _bus = None
    _meter = None
    _applied = False


def apply_profile(
    profile: str,
    *,
    task_queue_backend: str | None = None,
    billing_enabled: bool = False,
) -> dict[str, Any]:
    """Bind adapters for the given deploy profile.

    ``task_queue_backend`` overrides profile defaults: celery|eager|memory.
    """
    profile = (profile or "private").lower().strip()
    if profile not in {"lite", "private", "saas"}:
        logger.warning("Unknown DEPLOY_PROFILE=%r, falling back to private", profile)
        profile = "private"

    tq = (task_queue_backend or "").lower().strip()

    def _pick_meter() -> MeteringPort:
        if billing_enabled:
            from app.core.adapters.metering.sqlalchemy_meter import SqlAlchemyMeter

            return SqlAlchemyMeter()
        from app.core.adapters.metering.noop import NoopMeter

        return NoopMeter()

    if profile == "lite":
        from app.core.adapters.queue.eager_queue import EagerTaskQueue
        from app.core.adapters.queue.memory_queue import MemoryTaskQueue
        from app.core.adapters.cache.memory_only import MemoryOnlyCacheAdapter
        from app.core.adapters.bus.inprocess import InProcessEventBus

        if tq == "memory":
            queue: TaskQueuePort = MemoryTaskQueue()
        elif tq == "celery":
            from app.core.adapters.queue.celery_queue import CeleryTaskQueue

            queue = CeleryTaskQueue()
        else:
            queue = EagerTaskQueue()  # default lite
        cache: CachePort = MemoryOnlyCacheAdapter()
```

```
        bus: EventBusPort = InProcessEventBus()
        meter = _pick_meter()
    elif profile == "saas":
        from app.core.adapters.queue.celery_queue import CeleryTaskQueue
        from app.core.adapters.queue.eager_queue import EagerTaskQueue
        from app.core.adapters.queue.memory_queue import MemoryTaskQueue
        from app.core.adapters.cache.redis_l2 import RedisL2CacheAdapter
        from app.core.adapters.bus.redis_pubsub import RedisEventBus

        if tq == "eager":
            queue = EagerTaskQueue()
        elif tq == "memory":
            queue = MemoryTaskQueue()
        else:
            queue = CeleryTaskQueue()
        cache = RedisL2CacheAdapter()
        bus = RedisEventBus()
        # SaaS defaults to SQL meter when billing flag on
        meter = _pick_meter() if billing_enabled else _pick_meter()
    else:  # private
        from app.core.adapters.queue.celery_queue import CeleryTaskQueue
        from app.core.adapters.queue.eager_queue import EagerTaskQueue
        from app.core.adapters.queue.memory_queue import MemoryTaskQueue
        from app.core.adapters.cache.redis_l2 import RedisL2CacheAdapter
        from app.core.adapters.bus.redis_pubsub import RedisEventBus

        if tq == "eager":
            queue = EagerTaskQueue()
        elif tq == "memory":
            queue = MemoryTaskQueue()
        else:
            queue = CeleryTaskQueue()
        cache = RedisL2CacheAdapter()
        bus = RedisEventBus()
        meter = _pick_meter()

    bind_ports(queue=queue, cache=cache, bus=bus, meter=meter, profile=profile)
    summary = {
        "profile": profile,
        "task_queue": queue.backend_name,
        "cache": cache.backend_name,
        "event_bus": bus.backend_name,
        "metering": meter.backend_name,
        "eager": queue.is_eager(),
    }
    logger.info("Deploy profile applied: %s", summary)
    return summary


def apply_profile_from_settings() -> dict[str, Any]:
```

```
    """Read ``app.config.settings`` and apply profile (idempotent-ish)."""
    from app.config import settings

    profile = getattr(settings, "DEPLOY_PROFILE", None) or "private"
    # Auto-lite when eager + sqlite and profile not forced
    if str(profile).lower() == "auto":
        db = str(getattr(settings, "DATABASE_URL", "") or "")
        if getattr(settings, "CELERY_TASK_ALWAYS_EAGER", False) and "sqlite" in db:
            profile = "lite"
        else:
            profile = "private"

    tq = getattr(settings, "TASK_QUEUE_BACKEND", None) or ""
    # If private/saas but CELERY_TASK_ALWAYS_EAGER, prefer eager queue when backend empty
    if not tq and getattr(settings, "CELERY_TASK_ALWAYS_EAGER", False):
        if str(profile).lower() in {"private", "saas", "auto"}:
            tq = "eager"

    billing = bool(getattr(settings, "BILLING_ENABLED", False))
    return apply_profile(
        str(profile), task_queue_backend=tq or None, billing_enabled=billing
    )


def profile_status() -> dict[str, Any]:
    q = get_task_queue()
    return {
        "profile": current_profile(),
        "task_queue": q.backend_name,
        "cache": get_cache_port().backend_name,
        "event_bus": get_event_bus().backend_name,
        "metering": get_meter().backend_name,
        "eager": q.is_eager(),
        "applied": _applied,
    }
# === File: app/core/resilience/__init__.py ===
  # AgentFlow-Eval V1.0.0
"""Resilience primitives: retry, circuit breaker, timeout, fallback."""

from app.core.resilience.circuit_breaker import (
    CircuitBreaker,
    CircuitOpenError,
    CircuitState,
    get_breaker,
    reset_all_breakers,
)
from app.core.resilience.policy import (
    ResiliencePolicy,
    default_llm_policy,
    protected_call,
```

```
    protected_call_async,
)
from app.core.resilience.retry import (
    async_retrying,
    retry_async,
    retry_sync,
)
from app.core.resilience.timeout import (
    TimeoutExceededError,
    with_timeout,
    with_timeout_sync,
)

__all__ = [
    "CircuitBreaker",
    "CircuitOpenError",
    "CircuitState",
    "ResiliencePolicy",
    "TimeoutExceededError",
    "async_retrying",
    "default_llm_policy",
    "get_breaker",
    "protected_call",
    "protected_call_async",
    "reset_all_breakers",
    "retry_async",
    "retry_sync",
    "with_timeout",
    "with_timeout_sync",
]
# === File: app/core/resilience/circuit_breaker.py ===
  # AgentFlow-Eval V1.0.0
"""Async-safe circuit breaker (closed → open → half-open).

Semantics (aligned with common circuitbreaker / pybreaker defaults):
  - failure_threshold: consecutive failures before opening (default 5)
  - recovery_timeout: seconds in OPEN before allowing a probe (default 60)
  - half-open: one successful probe closes the circuit; any failure re-opens
"""

from __future__ import annotations

import logging
import threading
import time
from collections.abc import Awaitable, Callable
from enum import Enum
from typing import Any, TypeVar

logger = logging.getLogger(__name__)
```

```
T = TypeVar("T")


class CircuitState(str, Enum):
    """Circuit breaker states."""

    CLOSED = "closed"
    OPEN = "open"
    HALF_OPEN = "half_open"


class CircuitOpenError(RuntimeError):
    """Raised when a call is rejected because the circuit is open."""

    def __init__(self, name: str, recovery_remaining: float = 0.0) -> None:
        self.name = name
        self.recovery_remaining = recovery_remaining
        super().__init__(
            f"Circuit '{name}' is OPEN (retry in ~{recovery_remaining:.1f}s)"
        )


class CircuitBreaker:
    """Thread-safe circuit breaker for sync and async callables."""

    def __init__(
        self,
        name: str,
        *,
        failure_threshold: int = 5,
        recovery_timeout: float = 60.0,
        success_threshold: int = 1,
        excluded_exceptions: tuple[type[BaseException], ...] = (),
    ) -> None:
        """Initialize a named circuit breaker.

        Args:
            name: Breaker identity (used in logs/metrics).
            failure_threshold: Consecutive failures to open the circuit.
            recovery_timeout: Seconds to wait before half-open probe.
            success_threshold: Successes in half-open required to close.
            excluded_exceptions: Exceptions that do not count as failures.
        """
        if failure_threshold < 1:
            raise ValueError("failure_threshold must be >= 1")
        if recovery_timeout < 0:
            raise ValueError("recovery_timeout must be >= 0")

        self.name = name
```

```
        self.failure_threshold = int(failure_threshold)
        self.recovery_timeout = float(recovery_timeout)
        self.success_threshold = max(1, int(success_threshold))
        self.excluded_exceptions = excluded_exceptions

        self._state = CircuitState.CLOSED
        self._failure_count = 0
        self._success_count = 0
        self._opened_at: float | None = None
        self._lock = threading.RLock()

    # ---- state introspection ----

    @property
    def state(self) -> CircuitState:
        with self._lock:
            self._maybe_transition_to_half_open()
            return self._state

    @property
    def failure_count(self) -> int:
        with self._lock:
            return self._failure_count

    def reset(self) -> None:
        """Force circuit back to CLOSED (tests / admin)."""
        with self._lock:
            self._state = CircuitState.CLOSED
            self._failure_count = 0
            self._success_count = 0
            self._opened_at = None
        self._emit_state()

    def _maybe_transition_to_half_open(self) -> None:
        if self._state != CircuitState.OPEN or self._opened_at is None:
            return
        elapsed = time.monotonic() - self._opened_at
        if elapsed >= self.recovery_timeout:
            self._state = CircuitState.HALF_OPEN
            self._success_count = 0
            logger.info("Circuit '%s' → HALF_OPEN (probe allowed)", self.name)
            self._emit_state_unlocked()

    def _recovery_remaining(self) -> float:
        if self._state != CircuitState.OPEN or self._opened_at is None:
            return 0.0
        left = self.recovery_timeout - (time.monotonic() - self._opened_at)
        return max(0.0, left)

    def _before_call(self) -> None:
```

```
    with self._lock:
        self._maybe_transition_to_half_open()
        if self._state == CircuitState.OPEN:
            raise CircuitOpenError(self.name, self._recovery_remaining())

def _on_success(self) -> None:
    with self._lock:
        if self._state == CircuitState.HALF_OPEN:
            self._success_count += 1
            if self._success_count >= self.success_threshold:
                self._state = CircuitState.CLOSED
                self._failure_count = 0
                self._success_count = 0
                self._opened_at = None
                logger.info("Circuit '%s' → CLOSED (probe succeeded)", self.name)
                self._emit_state_unlocked()
        else:
            self._failure_count = 0
    self._record_result("success")

def _on_failure(self, exc: BaseException) -> None:
    if isinstance(exc, self.excluded_exceptions):
        return
    with self._lock:
        self._failure_count += 1
        if self._state == CircuitState.HALF_OPEN:
            self._trip_open()
        elif (
            self._state == CircuitState.CLOSED
            and self._failure_count >= self.failure_threshold
        ):
            self._trip_open()
    self._record_result("failure")

def _trip_open(self) -> None:
    self._state = CircuitState.OPEN
    self._opened_at = time.monotonic()
    self._success_count = 0
    logger.warning(
        "Circuit '%s' → OPEN after %d failures (recovery=%.0fs)",
        self.name,
        self._failure_count,
        self.recovery_timeout,
    )
    self._emit_state_unlocked()

def _emit_state(self) -> None:
    with self._lock:
        self._emit_state_unlocked()
```

```
def _emit_state_unlocked(self) -> None:
    try:
        from app.core.observability.metrics import observe_circuit_state

        observe_circuit_state(self.name, self._state.value)
    except Exception:
        pass

def _record_result(self, result: str) -> None:
    try:
        from app.core.observability.metrics import observe_circuit_call

        observe_circuit_call(self.name, result)
    except Exception:
        pass

# ---- execution ----

def call(self, fn: Callable[..., T], *args: Any, **kwargs: Any) -> T:
    """Execute a sync callable through the breaker."""
    self._before_call()
    try:
        result = fn(*args, **kwargs)
    except Exception as exc:
        self._on_failure(exc)
        raise
    self._on_success()
    return result

async def call_async(
    self,
    fn: Callable[..., Awaitable[T]],
    *args: Any,
    **kwargs: Any,
) -> T:
    """Execute an async callable through the breaker."""
    self._before_call()
    try:
        result = await fn(*args, **kwargs)
    except Exception as exc:
        self._on_failure(exc)
        raise
    self._on_success()
    return result

def __call__(self, fn: Callable[..., Any]) -> Callable[..., Any]:
    """Decorator for sync or async functions."""
    import functools
    import inspect
```

```
        if inspect.iscoroutinefunction(fn):

            @functools.wraps(fn)
            async def async_wrapper(*args: Any, **kwargs: Any) -> Any:
                return await self.call_async(fn, *args, **kwargs)

            return async_wrapper

        @functools.wraps(fn)
        def sync_wrapper(*args: Any, **kwargs: Any) -> Any:
            return self.call(fn, *args, **kwargs)

        return sync_wrapper


# Process-wide named breakers
_BREAKERS: dict[str, CircuitBreaker] = {}
_BREAKERS_LOCK = threading.Lock()


def get_breaker(
    name: str,
    *,
    failure_threshold: int | None = None,
    recovery_timeout: float | None = None,
) -> CircuitBreaker:
    """Get or create a process-wide named circuit breaker.

    Thresholds are read from settings when not provided.
    """
    with _BREAKERS_LOCK:
        if name in _BREAKERS:
            return _BREAKERS[name]
        ft, rt = failure_threshold, recovery_timeout
        if ft is None or rt is None:
            try:
                from app.config import settings

                if ft is None:
                    ft = int(getattr(settings, "CIRCUIT_FAILURE_THRESHOLD", 5))
                if rt is None:
                    rt = float(getattr(settings, "CIRCUIT_RECOVERY_TIMEOUT_SEC", 60.0))
            except Exception:
                ft = ft if ft is not None else 5
                rt = rt if rt is not None else 60.0
        breaker = CircuitBreaker(
            name,
            failure_threshold=ft,
            recovery_timeout=rt,
        )
```

```
        _BREAKERS[name] = breaker
        return breaker


def reset_all_breakers() -> None:
    """Reset all registered breakers (tests)."""
    with _BREAKERS_LOCK:
        for b in _BREAKERS.values():
            b.reset()
        _BREAKERS.clear()
# === File: app/core/resilience/policy.py ===
  # AgentFlow-Eval V1.0.0
"""Composed resilience policy: circuit + retry + timeout + fallback."""

from __future__ import annotations

import logging
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from typing import Any, TypeVar

from app.core.resilience.circuit_breaker import (
    CircuitBreaker,
    CircuitOpenError,
    get_breaker,
)
from app.core.resilience.retry import DEFAULT_RETRY_EXCEPTIONS, retry_async, retry_sync
from app.core.resilience.timeout import (
    TimeoutExceededError,
    with_timeout,
    with_timeout_sync,
)

logger = logging.getLogger(__name__)

T = TypeVar("T")


@dataclass(frozen=True)
class ResiliencePolicy:
    """Configuration bundle for protected external calls."""

    name: str = "llm"
    max_attempts: int = 3
    min_wait: float = 1.0
    max_wait: float = 10.0
    timeout_sec: float = 30.0
    failure_threshold: int = 5
    recovery_timeout: float = 60.0
    retry_exceptions: tuple[type[BaseException], ...] = DEFAULT_RETRY_EXCEPTIONS
```

```
    enable_circuit: bool = True
    enable_retry: bool = True


def default_llm_policy(name: str = "llm") -> ResiliencePolicy:
    """Build policy from application settings."""
    try:
        from app.config import settings

        return ResiliencePolicy(
            name=name,
            max_attempts=int(getattr(settings, "LLM_MAX_RETRIES", 3) or 3),
            min_wait=float(getattr(settings, "LLM_RETRY_MIN_WAIT_SEC", 1.0) or 1.0),
            max_wait=float(getattr(settings, "LLM_RETRY_MAX_WAIT_SEC", 10.0) or 10.0),
            timeout_sec=float(getattr(settings, "LLM_CALL_TIMEOUT_SEC", 30.0) or 30.0),
            failure_threshold=int(
                getattr(settings, "CIRCUIT_FAILURE_THRESHOLD", 5) or 5
            ),
            recovery_timeout=float(
                getattr(settings, "CIRCUIT_RECOVERY_TIMEOUT_SEC", 60.0) or 60.0
            ),
            enable_circuit=bool(getattr(settings, "CIRCUIT_ENABLED", True)),
            enable_retry=bool(getattr(settings, "LLM_RETRY_ENABLED", True)),
        )
    except Exception:
        return ResiliencePolicy(name=name)


def _record_fallback(name: str, reason: str) -> None:
    try:
        from app.core.observability.metrics import observe_fallback

        observe_fallback(name, reason)
    except Exception:
        pass


async def protected_call_async(
    fn: Callable[..., Awaitable[T]],
    *args: Any,
    policy: ResiliencePolicy | None = None,
    fallback: Callable[..., Awaitable[T] | T] | None = None,
    **kwargs: Any,
) -> T:
    """Execute an async call with circuit breaker, retry, timeout, and optional fallback.

    Flow:
      1. Circuit open? → fallback or raise CircuitOpenError
      2. Each attempt wrapped in timeout
      3. Transient failures retried (tenacity exponential backoff, max 3)
```

```
  4. Success / failure recorded on circuit breaker
  5. Exhausted / open → invoke fallback if provided

Args:
    fn: Async callable.
    policy: Resilience knobs; defaults to LLM settings.
    fallback: Sync or async callable used on terminal failure.
"""
pol = policy or default_llm_policy()
breaker: CircuitBreaker | None = None
if pol.enable_circuit:
    breaker = get_breaker(
        pol.name,
        failure_threshold=pol.failure_threshold,
        recovery_timeout=pol.recovery_timeout,
    )

async def _one_attempt() -> T:
    return await with_timeout(
        fn(*args, **kwargs),
        timeout_sec=pol.timeout_sec,
        name=pol.name,
    )

async def _run() -> T:
    if pol.enable_retry:
        return await retry_async(
            _one_attempt,
            name=pol.name,
            max_attempts=pol.max_attempts,
            min_wait=pol.min_wait,
            max_wait=pol.max_wait,
            retry_exceptions=pol.retry_exceptions + (TimeoutExceededError,),
        )
    return await _one_attempt()

try:
    if breaker is not None:
        return await breaker.call_async(_run)
    return await _run()
except CircuitOpenError as exc:
    logger.warning("Protected call '%s' short-circuited: %s", pol.name, exc)
    if fallback is not None:
        _record_fallback(pol.name, "circuit_open")
        return await _invoke_fallback_async(fallback, *args, **kwargs)
    raise
except Exception as exc:
    logger.warning("Protected call '%s' failed: %s", pol.name, exc)
    if fallback is not None:
        reason = (
```

```
                "timeout"
                if isinstance(exc, (TimeoutError, TimeoutExceededError))
                else "error"
            )
            _record_fallback(pol.name, reason)
            return await _invoke_fallback_async(fallback, *args, **kwargs)
        raise


def protected_call(
    fn: Callable[..., T],
    *args: Any,
    policy: ResiliencePolicy | None = None,
    fallback: Callable[..., T] | None = None,
    **kwargs: Any,
) -> T:
    """Sync variant of :func:`protected_call_async`."""
    pol = policy or default_llm_policy()
    breaker: CircuitBreaker | None = None
    if pol.enable_circuit:
        breaker = get_breaker(
            pol.name,
            failure_threshold=pol.failure_threshold,
            recovery_timeout=pol.recovery_timeout,
        )

    def _one_attempt() -> T:
        return with_timeout_sync(
            fn,
            *args,
            timeout_sec=pol.timeout_sec,
            name=pol.name,
            **kwargs,
        )

    def _run() -> T:
        if pol.enable_retry:
            return retry_sync(
                _one_attempt,
                name=pol.name,
                max_attempts=pol.max_attempts,
                min_wait=pol.min_wait,
                max_wait=pol.max_wait,
                retry_exceptions=pol.retry_exceptions + (TimeoutExceededError,),
            )
        return _one_attempt()

    try:
        if breaker is not None:
            return breaker.call(_run)
```

```
        return _run()
    except CircuitOpenError as exc:
        logger.warning("Protected call '%s' short-circuited: %s", pol.name, exc)
        if fallback is not None:
            _record_fallback(pol.name, "circuit_open")
            return fallback(*args, **kwargs)
        raise
    except Exception as exc:
        logger.warning("Protected call '%s' failed: %s", pol.name, exc)
        if fallback is not None:
            reason = (
                "timeout"
                if isinstance(exc, (TimeoutError, TimeoutExceededError))
                else "error"
            )
            _record_fallback(pol.name, reason)
            return fallback(*args, **kwargs)
        raise


async def _invoke_fallback_async(
    fallback: Callable[..., Awaitable[T] | T],
    *args: Any,
    **kwargs: Any,
) -> T:
    import inspect

    result = fallback(*args, **kwargs)
    if inspect.isawaitable(result):
        return await result  # type: ignore[misc]
    return result  # type: ignore[return-value]
# === File: app/core/resilience/retry.py ===
  # AgentFlow-Eval V1.0.0
"""Retry helpers built on tenacity (exponential backoff)."""

from __future__ import annotations

import logging
from collections.abc import Awaitable, Callable
from typing import Any, TypeVar

from tenacity import (
    AsyncRetrying,
    RetryCallState,
    RetryError,
    Retrying,
    retry_if_exception_type,
    stop_after_attempt,
    wait_exponential,
)
```

```
logger = logging.getLogger(__name__)

T = TypeVar("T")


def _collect_retry_exceptions() -> tuple[type[BaseException], ...]:
    """Build the default set of transient exception types to retry."""
    import asyncio

    errs: list[type[BaseException]] = [TimeoutError, ConnectionError, OSError]
    # On 3.11+ asyncio.TimeoutError is TimeoutError; keep both for safety
    if asyncio.TimeoutError is not TimeoutError:  # type: ignore[comparison-overlap]
        errs.append(asyncio.TimeoutError)
    try:
        import httpx

        errs.extend((httpx.TimeoutException, httpx.TransportError))
    except ImportError:
        pass
    try:
        import openai

        for name in ("APITimeoutError", "APIConnectionError", "RateLimitError"):
            cls = getattr(openai, name, None)
            if isinstance(cls, type) and issubclass(cls, BaseException):
                errs.append(cls)
    except ImportError:
        pass
    # Deduplicate while preserving order
    seen: set[type[BaseException]] = set()
    unique: list[type[BaseException]] = []
    for e in errs:
        if e not in seen:
            seen.add(e)
            unique.append(e)
    return tuple(unique)


# Transient errors typical of LLM / HTTP clients
DEFAULT_RETRY_EXCEPTIONS: tuple[type[BaseException], ...] = _collect_retry_exceptions()


def _load_retry_settings() -> tuple[int, float, float]:
    try:
        from app.config import settings

        attempts = int(getattr(settings, "LLM_MAX_RETRIES", 3) or 3)
        min_wait = float(getattr(settings, "LLM_RETRY_MIN_WAIT_SEC", 1.0) or 1.0)
        max_wait = float(getattr(settings, "LLM_RETRY_MAX_WAIT_SEC", 10.0) or 10.0)
```

```
        return max(1, attempts), min_wait, max_wait
    except Exception:
        return 3, 1.0, 10.0


def _before_sleep(name: str) -> Callable[[RetryCallState], None]:
    def _cb(state: RetryCallState) -> None:
        exc = state.outcome.exception() if state.outcome else None
        logger.warning(
            "Retry %s attempt=%s next_wait=%.2fs error=%s",
            name,
            state.attempt_number,
            state.next_action.sleep if state.next_action else 0,
            exc,
        )
        try:
            from app.core.observability.metrics import observe_retry

            observe_retry(name, attempt=state.attempt_number)
        except Exception:
            pass

    return _cb


def async_retrying(
    *,
    name: str = "llm",
    max_attempts: int | None = None,
    min_wait: float | None = None,
    max_wait: float | None = None,
    retry_exceptions: tuple[type[BaseException], ...] | None = None,
) -> AsyncRetrying:
    """Build an ``AsyncRetrying`` controller with exponential backoff."""
    attempts, d_min, d_max = _load_retry_settings()
    return AsyncRetrying(
        stop=stop_after_attempt(max_attempts or attempts),
        wait=wait_exponential(
            multiplier=1,
            min=min_wait if min_wait is not None else d_min,
            max=max_wait if max_wait is not None else d_max,
        ),
        retry=retry_if_exception_type(retry_exceptions or DEFAULT_RETRY_EXCEPTIONS),
        before_sleep=_before_sleep(name),
        reraise=True,
    )


def sync_retrying(
    *,
```

```
    name: str = "llm",
    max_attempts: int | None = None,
    min_wait: float | None = None,
    max_wait: float | None = None,
    retry_exceptions: tuple[type[BaseException], ...] | None = None,
) -> Retrying:
    """Build a sync ``Retrying`` controller."""
    attempts, d_min, d_max = _load_retry_settings()
    return Retrying(
        stop=stop_after_attempt(max_attempts or attempts),
        wait=wait_exponential(
            multiplier=1,
            min=min_wait if min_wait is not None else d_min,
            max=max_wait if max_wait is not None else d_max,
        ),
        retry=retry_if_exception_type(retry_exceptions or DEFAULT_RETRY_EXCEPTIONS),
        before_sleep=_before_sleep(name),
        reraise=True,
    )


async def retry_async(
    fn: Callable[..., Awaitable[T]],
    *args: Any,
    name: str = "llm",
    max_attempts: int | None = None,
    min_wait: float | None = None,
    max_wait: float | None = None,
    retry_exceptions: tuple[type[BaseException], ...] | None = None,
    **kwargs: Any,
) -> T:
    """Run an async callable with tenacity exponential backoff retries."""
    controller = async_retrying(
        name=name,
        max_attempts=max_attempts,
        min_wait=min_wait,
        max_wait=max_wait,
        retry_exceptions=retry_exceptions,
    )
    async for attempt in controller:
        with attempt:
            return await fn(*args, **kwargs)
    raise RuntimeError("retry_async exhausted without result")  # pragma: no cover


def retry_sync(
    fn: Callable[..., T],
    *args: Any,
    name: str = "llm",
    max_attempts: int | None = None,
```

```
    min_wait: float | None = None,
    max_wait: float | None = None,
    retry_exceptions: tuple[type[BaseException], ...] | None = None,
    **kwargs: Any,
) -> T:
    """Run a sync callable with tenacity exponential backoff retries."""
    controller = sync_retrying(
        name=name,
        max_attempts=max_attempts,
        min_wait=min_wait,
        max_wait=max_wait,
        retry_exceptions=retry_exceptions,
    )
    for attempt in controller:
        with attempt:
            return fn(*args, **kwargs)
    raise RuntimeError("retry_sync exhausted without result")  # pragma: no cover


__all__ = [
    "DEFAULT_RETRY_EXCEPTIONS",
    "RetryError",
    "async_retrying",
    "retry_async",
    "retry_sync",
    "sync_retrying",
]
# === File: app/core/resilience/timeout.py ===
  # AgentFlow-Eval V1.0.0
"""Timeout wrappers for sync and async callables."""

from __future__ import annotations

import asyncio
import concurrent.futures
import logging
from collections.abc import Awaitable, Callable
from typing import Any, TypeVar

logger = logging.getLogger(__name__)

T = TypeVar("T")


class TimeoutExceededError(TimeoutError):
    """Raised when an operation exceeds its configured timeout."""

    def __init__(self, name: str, timeout_sec: float) -> None:
        self.name = name
        self.timeout_sec = timeout_sec
```

```
        super().__init__(f"'{name}' timed out after {timeout_sec}s")


def _default_timeout() -> float:
    try:
        from app.config import settings

        return float(getattr(settings, "LLM_CALL_TIMEOUT_SEC", 30.0) or 30.0)
    except Exception:
        return 30.0


def _record_timeout(name: str) -> None:
    try:
        from app.core.observability.metrics import observe_timeout

        observe_timeout(name)
    except Exception:
        pass


async def with_timeout(
    awaitable: Awaitable[T],
    *,
    timeout_sec: float | None = None,
    name: str = "operation",
) -> T:
    """Await ``awaitable`` with a deadline (default 30s from settings)."""
    limit = timeout_sec if timeout_sec is not None else _default_timeout()
    if limit <= 0:
        return await awaitable
    try:
        return await asyncio.wait_for(awaitable, timeout=limit)
    except asyncio.TimeoutError as exc:
        _record_timeout(name)
        logger.warning("Timeout: %s after %.1fs", name, limit)
        raise TimeoutExceededError(name, limit) from exc


def with_timeout_sync(
    fn: Callable[..., T],
    *args: Any,
    timeout_sec: float | None = None,
    name: str = "operation",
    **kwargs: Any,
) -> T:
    """Run a sync callable in a worker thread with a deadline."""
    limit = timeout_sec if timeout_sec is not None else _default_timeout()
    if limit <= 0:
        return fn(*args, **kwargs)
```

```
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
    future = pool.submit(fn, *args, **kwargs)
    try:
        return future.result(timeout=limit)
    except concurrent.futures.TimeoutError as exc:
        future.cancel()
        _record_timeout(name)
        logger.warning("Timeout: %s after %.1fs", name, limit)
        raise TimeoutExceededError(name, limit) from exc
```